

Performance Modeling of a KV-Stationary Systolic Array

Hudson Strauss & Dhruv Patel

High-Performance Computer Architecture | May 2026

MOTIVATION: THE MEMORY WALL

Transformer inference is increasingly **memory-bound** during autoregressive decoding.

- Each token generation requires full access to the Key-Value (KV) cache.
- Memory traffic scales linearly with context length.
- **MQA (Multi-Query Attention)** reduces footprint by sharing KV across query heads.
- Despite footprint reduction, standard accelerators still treat attention as generic matrix multiplication.

USE CASE: Edge Cyber-Physical Anomaly Detection



Real-time Monitoring

The KV-stationary design is ideal for streaming sensor data where memory and power are constrained:

- Input: streaming sensor, telemetry, or network sequences
- Task: detect anomalies using long-range temporal context
- Benefit: KV-stationary MQA enables local transformer inference with reduced KV-cache traffic and lower memory movement

THE SYSTOLIC MISMATCH

Standard Dataflows

Weight, Output, and Input stationary mappings are optimized for dense GEMM patterns, not the streaming nature of token decoding.

Hardware Inefficiency

Traditional systolic arrays suffer from excessive memory movement when keys/values are re-read for every incoming query vector.

The Hypothesis

A specialized **KV** dataflow will improve PE utilization and lower latency by keeping data resident near compute.

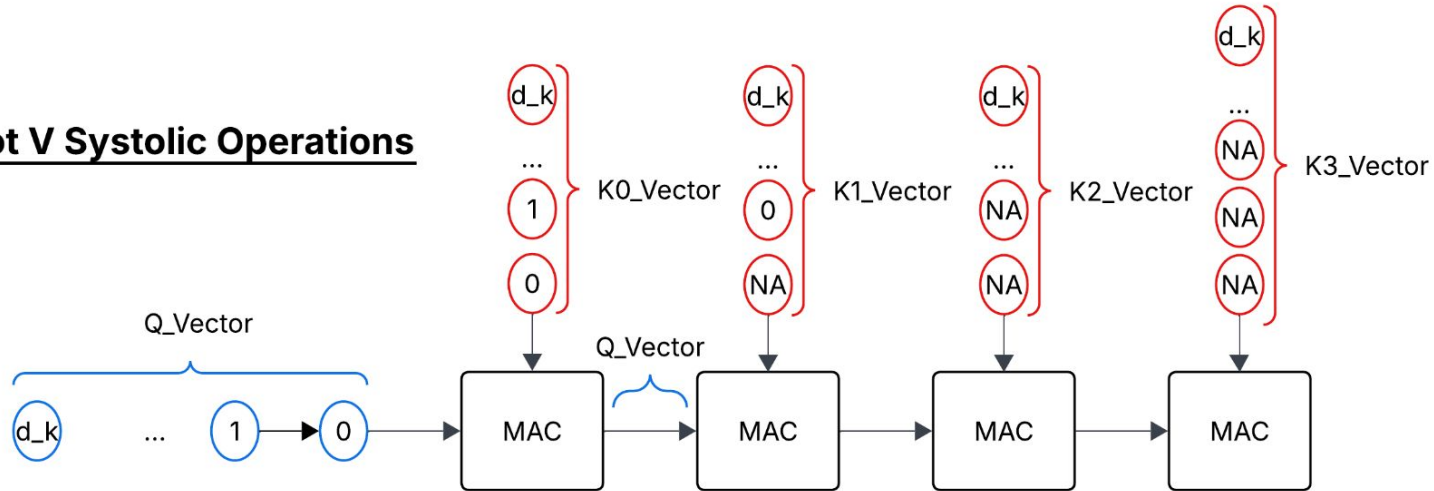
DESIGN: KV MAPPING

In our proposed dataflow, the systolic array is reconfigured specifically for the MQA decoding phase:

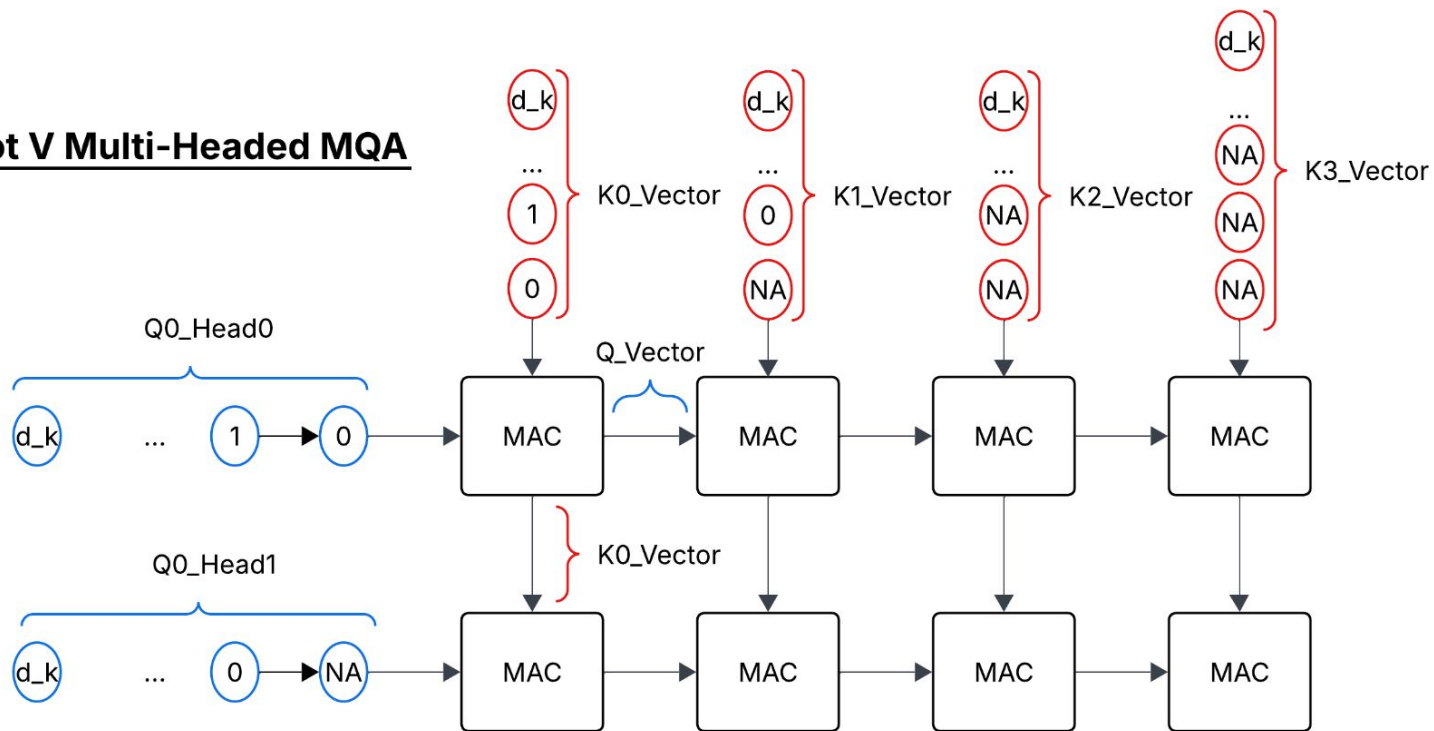
- **Stationary KVs:** Each Processing Element (PE) holds a resident Key (K) and Value (V) pair.
- **Streaming Queries:** Query vectors (Q) flow across the fabric, interacting with stationary K/V data.
- **Fused Softmax:** Partial sum and max variables propagate vertically, enabling online softmax computation without external memory stalls.

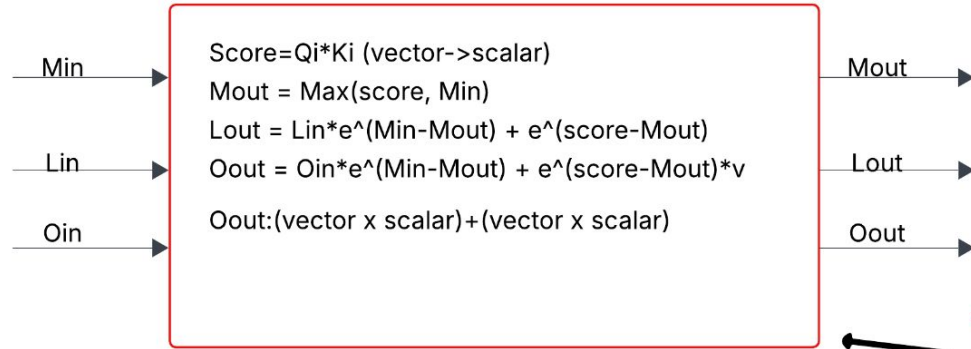
Demystify “Stationary” K

K dot V Systolic Operations

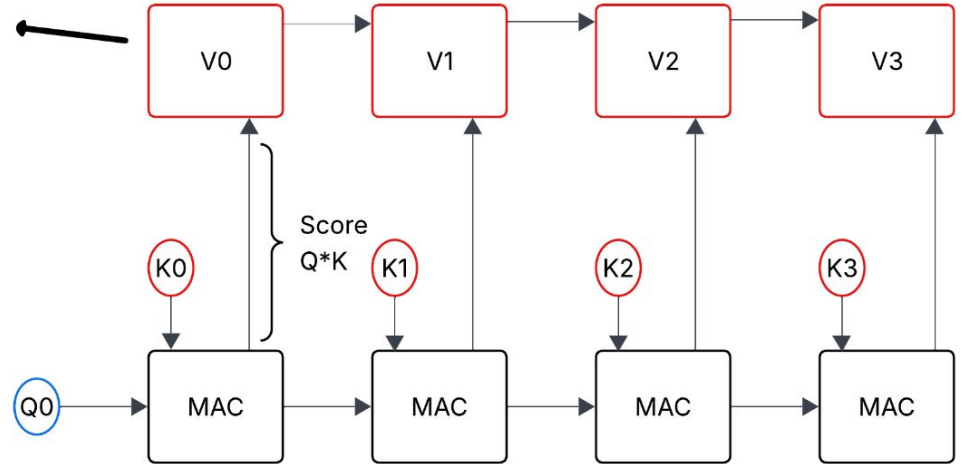


K dot V Multi-Headed MQA

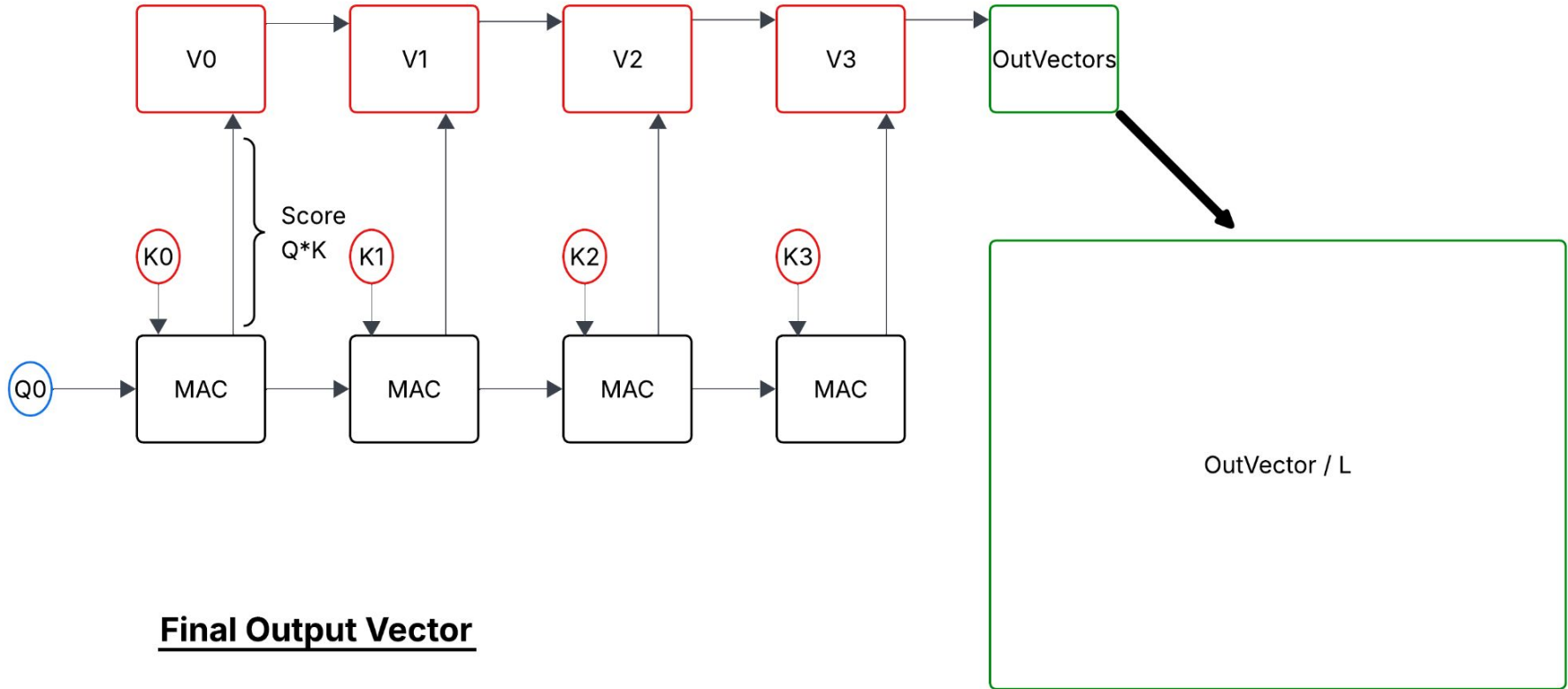




Running Attention PE



Ideal Total cycles $\approx (2N-1)X$ X: Cycle time of red PE. N: token length



Serialization Concern

In attention you typically normalize the vectors with a Softmax pass which requires a Max and a second pass through the vector.

$$\text{softmax}(x_i) = \frac{e^{x_i - m}}{\sum_{j=1}^n e^{x_j - m}}$$

Online Softmax Normalization is Abstracted in CUDA however it enables pipelining

$$m_{out} = \max(m_{in}, s_i)$$

$$l_{out} = l_{in} e^{(m_{in} - m_{out})} + e^{(s_i - m_{out})}$$

$$O_{out} = O_{in} e^{(m_{in} - m_{out})} + V_i e^{(s_i - m_{out})}$$

EXPERIMENTAL PLAN

Workload Sweeps

Sequence lengths: 128 to 1024 tokens.

Head dimension: 32, 64, 128.

Batch size: 1 (Edge focus)

Layer Count: 1

Hardware Sweeps

Array size: 16 x 16 to 64 x 64

Memory BW constraints.

Precision: INT8

Proposed Metrics

Latency

Throughput

KV Traffic Reduction,

PE Utilization

ANTICIPATED SCALING PERFORMANCE



Justification:

MQA Accelerator Track Fit: #1 + #4

This project fits **Track #1** because it co-designs the MQA attention algorithm with a specialized systolic-array dataflow, using KV-stationary execution, query streaming, and in-flight online softmax accumulation $(m,l,O)(m,l,O)$ to reduce memory movement and sequence latency.

It also fits **Track #4** if extended to a multi-array/chiplet model, where heads, query blocks, or layers are distributed across multiple systolic cores while analyzing interconnect traffic, KV sharing, and scaling bottlenecks.

PROJECT ROADMAP

Project Phases & Milestones			
Week 7	Week 8 (Mid)	Week 9	Week 10
Baseline Performance Modeling & Setup	KV-Stationary Simulator Implementation	Parameter Sweeps & Comparative Analysis	Final Report & Performance Validation

Q & A

Thank you for your attention.

Hudson Strauss & Dhruv Patel

Performance Modeling of KV-Stationary Systolic Arrays